PYSPARK & SPARK SQL

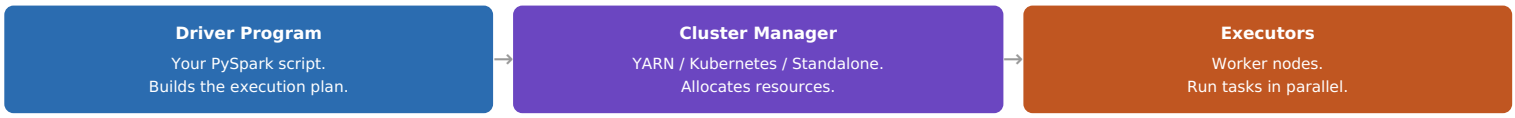
A Practical Guide to Both Coding Styles

DataFrame API spark.sql()

1 What Is Spark, PySpark & Spark SQL?

Apache Spark is a distributed computing engine for processing large datasets across a cluster of machines. PySpark is its Python API, and Spark SQL is the module that lets you query data using standard SQL syntax — both run on the exact same execution engine.

The Big Picture



Two Ways to Write the Same Logic

Spark gives you a choice of **style**, not two different engines. Both compile down to the same Catalyst optimizer and physical execution plan:

- ✓ **PySpark DataFrame API** — method chaining in Python: `.filter().groupBy().agg()`
- ✓ **Spark SQL** — plain SQL strings executed with `spark.sql("...")` against registered views

KEY INSIGHT

Because both styles share the same Catalyst optimizer, a PySpark DataFrame chain and an equivalent SQL query usually produce **identical execution plans** and identical performance. The choice is about readability and team preference, not speed.

Starting a Session

PYSPARK

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
    .appName("MyApp")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate())

df = spark.read.csv("sales.csv", header=True, inferSchema=True)
```

2 Core Operations, Side by Side

The same five operations, shown in both styles. Registering a temp view is the bridge between the two worlds.

BRIDGE: DATAFRAME → SQL VIEW

```
df.createOrReplaceTempView("sales")
# now "sales" can be queried with spark.sql(...)
```

Select & Filter

PySpark	Spark SQL

`df.select("name","amount")`
`  .filter(df.amount > 100)`
	`SELECT name, amount`
	`FROM sales`
	`WHERE amount > 100`

Group & Aggregate

PySpark	Spark SQL

`df.groupBy("region")`
`  .agg(F.sum("amount")`
`  .alias("total"))`
	`SELECT region, SUM(amount) AS total`
	`FROM sales`
	`GROUP BY region`

Join

PySpark	Spark SQL

`orders.join(customers,`
`  on="cust_id", how="left")`
	`SELECT * FROM orders o`
	`LEFT JOIN customers c`
	`  ON o.cust_id = c.cust_id`

Add / Transform Column

PySpark	Spark SQL
`df.withColumn("tax",` `  F.col("amount") * 0.14)`	`SELECT *, amount * 0.14 AS tax` `FROM sales`

Sort & Limit

PySpark	Spark SQL

`df.orderBy(F.desc("amount"))`
`  .limit(10)`
	`SELECT * FROM sales`
	`ORDER BY amount DESC`
	`LIMIT 10`

3 Window Functions

Window functions compute a value across a set of rows related to the current row, without collapsing the rows like `GROUP BY` does. This is the highest-leverage feature to master in both styles.

PySpark Style

PYSPARK

```
from pyspark.sql import Window
import pyspark.sql.functions as F

w = Window.partitionBy("region").orderBy(F.desc("amount"))

df.withColumn("rank", F.rank().over(w)) \
  .withColumn("running_total",
      F.sum("amount").over(
          w.rowsBetween(Window.unboundedPreceding, Window.currentRow)))
```

Spark SQL Style

SPARK SQL

```
SELECT *,
  RANK() OVER (PARTITION BY region ORDER BY amount DESC) AS rank,
  SUM(amount) OVER (
      PARTITION BY region ORDER BY amount DESC
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total
FROM sales
```

NOTICE

The Spark SQL version reads almost line-for-line like the PySpark version — `Window.partitionBy().orderBy()` maps directly to `PARTITION BY ... ORDER BY`. Once you know one, the other is just a syntax translation.

Common Window Functions

`ROW_NUMBER()` `RANK()` `DENSE_RANK()` `LAG(col, n)` `LEAD(col, n)` `NTILE(n)` `SUM() OVER` `AVG() OVER` `FIRST_VALUE()` `LAST_VALUE()`

4 CTEs, Subqueries & Multi-Step Logic

SQL uses `WITH` clauses (CTEs) to break logic into named steps. PySpark achieves the same readability by chaining intermediate DataFrames.

SPARK SQL — CTE

```sql
WITH regional_totals AS (
  SELECT region, SUM(amount) AS total
  FROM sales
  GROUP BY region
)
SELECT *
FROM regional_totals
WHERE total > 10000
ORDER BY total DESC
```

PYSPARK — EQUIVALENT

```python
regional_totals = (df.groupBy("region")
    .agg(F.sum("amount").alias("total")))

result = (regional_totals
    .filter(F.col("total") > 10000)
    .orderBy(F.desc("total")))
```

STYLE NOTE

Deeply nested SQL subqueries can get hard to read. Many teams use CTEs in SQL, or chain named intermediate variables in PySpark — both achieve the same "readable pipeline" goal.

5 Function Reference Cheat-Sheet

pyspark.sql.functions ↔ SQL equivalents

PySpark (import as F)	Spark SQL
`F.col("x")`	`x`
`F.lit(5)`	`5`
`F.when(cond, a).otherwise(b)`	`CASE WHEN cond THEN a ELSE b END`
`F.coalesce(col1, col2)`	`COALESCE(col1, col2)`
`F.to_date("d", "yyyy-MM-dd")`	`TO_DATE(d, 'yyyy-MM-dd')`
`F.datediff(end, start)`	`DATEDIFF(end, start)`
`F.concat_ws("-", a, b)`	`CONCAT_WS('-', a, b)`
`F.explode("array_col")`	`LATERAL VIEW explode(array_col)`
`F.regexp_extract(c, pat, 1)`	`REGEXP_EXTRACT(c, pat, 1)`
`df.dropDuplicates(["id"])`	`SELECT DISTINCT ... / ROW_NUMBER() filter`

Running SQL Directly from PySpark

MIXING BOTH STYLES FREELY

```
df.createOrReplaceTempView("sales")

result = spark.sql("""
    SELECT region, SUM(amount) AS total
    FROM sales
    GROUP BY region
    HAVING SUM(amount) > 10000
""")

# result is a normal DataFrame — keep chaining PySpark methods
result.orderBy(F.desc("total")).show()
```

6 Performance Essentials

Do

- ✓ Use `.cache()` when a DataFrame is reused multiple times
- ✓ Broadcast small tables in joins: `F.broadcast(small_df)`
- ✓ Filter and select early to reduce shuffle size
- ✓ Use `.explain(True)` to inspect the physical plan
- ✓ Prefer built-in functions over Python UDFs

Avoid

- ✓ Calling `.collect()` on large DataFrames
- ✓ Python UDFs when a native function exists
- ✓ Unnecessary `.repartition()` calls
- ✓ Wide transformations without checking partition count

BROADCAST JOIN EXAMPLE

```
from pyspark.sql.functions import broadcast

orders.join(broadcast(small_lookup_df), on="code", how="left")
```

RULE OF THUMB

If you find yourself writing the same logic comfortably in SQL, write it in SQL. If you need Python control flow, loops, or reusable functions around your pipeline, reach for the DataFrame API. Mixing both in one pipeline is completely normal.

Quick Decision Guide

Choose PySpark DataFrame API when...	Choose Spark SQL when...
Building reusable, parameterized pipelines	The team is SQL-native or migrating existing queries
Logic needs Python conditionals / loops	Ad-hoc exploration in a notebook
Integrating with ML libraries / UDFs	Query is naturally expressed declaratively

⚡ End of guide — both styles compile to the same Catalyst execution plan.